

94-eos-sys-dev-dbs • EOS 系统设计产品数据分解结构

本文定义 EOS 系统设计各阶段输出产品数据文档的**分解结构**——即以**树形结构**表达的逐层分解。

版本：v1.38 | 修订：2026-09-21

一、产品数据及其形态

本文各章分别定义**一份产品数据**的分解结构（树）。本章是全篇共用的前提——产品数据清单与所在空间、各树成立的依据、每份产品数据的三个形态、形态之间的承接。未定处标【待裁】，待相应树补齐后再回填。

1.1 EOS 的产品数据清单

EOS 设计线以「需求—方案结对」逐对生成产品数据；结对之前另有一条**原始需求入口加工链**，把外部材料加工成第一份需求产品数据。开发过程应输出下列产品数据：

产品数据	文档（EOS 实例）	空间	本文
原始需求 †	00-origin-requirement-materials/10-raw-files/（任意载体）	—	\$2.0
文本化的原始需求 †	*.textualized.md	—	\$2.0
切分后的原始需求 †	*.chunk-{seq}.md（status=raw）	业务	\$2.0
澄清后的原始需求 †	同上文件（status=clarified）	业务	\$2.0
相关方需求	01-eos-specified-requirements.md	业务	\$2.1
职责视角业务流程概要	02-eos-business-role-process.md	业务	\$2.2
产品开发视角业务流程概要	03-eos-business-product-process.md	业务	\$2.2
职责视角业务流程定义	04-eos-business-detailed.md	业务	\$2.2
系统需求框架	05-eos-system-requirement-summary-architecture.md	方案	\$2.3
系统需求概要	06-eos-system-requirement-detailed.md	方案	\$2.3
平台构成框架	07-eos-platform-product-architecture.md	方案	\$2.4

产品数据	文档 (EOS 实例)	空间	本文
平台构成概要	(文档待定义)	方案	§2.4
配置方案	(文档待定义)	方案	§2.5
前后端组件方案	(文档待定义)	方案	§2.6

名称自带形态： ...框架 / ...概要 / ...定义 各对应一级形态——同一份产品数据按停在哪一级各占一行，同指一节。**恒定的是视角与业务对象那一截**（如「产品开发视角业务流程」），**可变的是形态那一截**（概要 / 定义，随停在哪个形态变）；跨形态指这份产品数据时用恒定截，指某一份文档时写全。† 四行是**同一份产品数据的入口加工形态**（一份材料加工到多细的程度序列），不是四份产品数据。**两处例外：** 01 相关方需求不分形态；**配置方案 / 前后端组件方案**的名称不含形态标记，其形态归属【待裁】。**业务流程**停在概要一级的文档分两个视角各一份——02 职责视角业务流程概要、03 产品开发视角业务流程概要；更深的 04 定义只有一份，且只做职责视角，见 §2.2。

「方案」是伞词——框架、概要、定义都是方案，只是详细程度不同；名称里的"方案"不指某一形态，故 配置方案 / 前后端组件方案 的形态归属不因名称而可判。表中 相关方需求 的全称是「**规范化的相关方需求**」（来源 = 相关方、加工程度 = 规范化）。

两个空间——上表"空间"列的含义：业务空间里的树回答"谁做什么、产出什么"，先不涉及由哪套电脑系统来实现；方案空间里的树写"这套承接系统要实现什么、由什么构成"（本设计实例中该承接系统 = EOS 平台，企业运营体系的信息化形态）。

空间标签是成对相对而言的，不是全篇的属性：**系统需求**相对**业务流程**是"方案"（写系统要实现什么），相对**平台构成**又是"需求"（平台构成写这套系统的软件构成）。"业务/方案"只说一对之内谁写问题、谁写解，不指文档等级。

两个视角下的业务流程

业务流程可以有**两个视角树**——**R 树**（职责视角，roles & responsibilities）与 **P 树**（产品开发视角，product or service）。下标索引的是所指产品本身：0 = 平台本身， $n \geq 1$ = 基于平台引擎构建的各个业务。

下标	所指产品	R 树（职责视角）	P 树（产品开发视角）
0	平台本身	R0 树（角色 = 业务配置角色）	P0 树（构件 = 引擎，单分支只有配置单元）
$n \geq 1$	基于平台引擎构建的业务	Rn 树（业务运行角色按职责完成业务对象闭环）	Pn 树（构件经 WBS 产出构件产品数据）

两族所用的视角名：R 树沿职责视角（SL）展开、P 树沿产品开发视角（PL）展开。

1.2 各产品数据树成立的理论依据

本文全篇建立在三条理论依据之上——每份产品数据可以条目化、可以树形化、可以由需求设计方案 N:1 分配。

依据一·为什么产品数据可以条目化

每份产品数据都必须能拆到**条目**。理由：设计的每一步产物都要能被**独立判定"是否被满足"**，而通行的需求书写标准正是这样要求单条需求的——**单数、可验证、唯一解释**（ISO/IEC/IEEE 29148 的良构需求特性，其中 **Singular** 一名即"一条只讲一件事"）。条目的判定标准（分解终止标准）引[通规范 §2.2.2](#)。

依据二·为什么产品数据可以树形化

因为**分层是复杂系统的常态结构**（Simon 的层级与近可分解性），且工程上早有分解纪律（WBS 的 **100% 规则**，即 MECE）。但要注意：**"能组树"不是自动的**——同一层的分组必须同时满足三条，否则得到的不是树：

#	条件	不满足则
一	判据单维 ——同一层的分组只用一个判据（如"系统职责"），不得混用两个（如职责与业务域并用）	同层内出现 交叉分类 → 得到的是 格 ，不是树
二	条目单归属 ——每条只归一个父节点（单继承）	得到的是 有向无环图（DAG） （多层级），不是树
三	分组完备互斥 ——子层完整覆盖父层、且互不重叠	出现 缺口或重叠 → 不是合格的树

本文各树均满足此三条。

依据三·为什么由需求设计方案可以 N:1 分配

因为设计被建模为**从需求域到方案域的映射**（公理化设计的 $FR \leftrightarrow DP$ 映射）。这条映射的形状由两端与一条约束决定：

- **定义域取需求侧的原子（叶）**——粗粒度节点**不能被 1:1 分配**：它内部含着多条性质不同的需求，交出去后无法判定谁负责哪一部分；
- **值域取方案侧当前可负责的实体（枝梢）**——这一刻**方案侧的叶尚未产出**（方案只到框架），方案侧能提供的最深实体就是架构末级节点；
- **映射必须是函数**——函数允许多对一（N:1：一个实体承担一族需求 = 内聚），**不允许一对多**（1:N：一条需求散在多个实体 = 变更传播与重复实现）。注意**两侧理由不同源**。

本链上真正用到这条分配的只有 04 → 05 与 06 → 07；01 → 02 与 02 → 03 两处，两侧的叶是同一批节点，不涉及映射，见 §1.4。

1.3 产品数据的三个形态

每份产品数据都是**同一棵树**，按定义到多深分**框架、概要、定义**三个形态。级语义术语（全树通用）：

术语	含义
叶 / 叶节点	树的 末级节点
枝梢 / 枝梢节点	框架的最深一级节点
干枝 / 干枝节点	树形结构上的 非叶节点 ——叶以外的各级
框架	根 → 枝梢 ——叶只以枝梢分解源判据的展开项露名字
概要	根 → 叶 ，各级节点只到 说明
定义	概要 + 各节点的详细定义

三形态由三级设计动作产出：**框架设计**出框架、**概要设计**出概要、**详细定义**出定义。**相邻两级可以合做**——一个设计动作一次交出两级（框架与概要合、概要与定义合），三级也可以一手做完；**也可以按需省略或裁剪**：某份产品数据不必每一级都停。

框架有阶：枝梢取哪一级由该次框架设计定，取越浅的级框架阶越高。「**枝梢**」始终指该次框架的**最深一级**，不是某个固定的树位——同一份业务流程产品数据可由浅到深给出各阶框架：**构件级高阶框架**（枝梢 = 产品构件）、**文档级高阶框架**（枝梢 = 业务文档），每一阶都是一份合法的框架。

叶的三种身份：在框架里只是枝梢上的一条展开项（名称级）；到概要才成为树节点（说明级）；到定义才带详细定义。

分解源判据附着于本次形态的最深一级：框架停在某一阶，判据就挂哪一级——停在文档级，判据挂业务文档、展开项列下一级名；停在任务节点级，判据挂任务节点、展开项列叶名。**到概要，叶已是树节点，判据的内容即叶本身，最深一级不再另设展开清单。**

叶不只是“最深的那一级”——它有判据：**叶必须切到可被独立解释、独立验证、可被唯一交出去的粒度**。所以“某棵树到哪一级为止”是一个**判断**，不是一个位置约定；判定标准（分解终止标准）见[通用规范 §2.2.2](#)。

1.4 形态之间的承接与已定的树间对应

承接规则：上一份产品数据的**最深可用原子（叶）**，对到下一份产品数据**能负责这条语义的节点——干枝或枝梢，以语义定级**；分配关系不变。

- **同一份产品数据的形态之间不是承接**：框架、概要、定义三形态之间（05↔06）是形态轴上的深浅，不在此列。本链上的承接只有两处：04→05、06→07。
- **叶由上游直接给出的步，都不是承接**：01 → 02 与 02 → 03 两处，两侧的 ⑤ ⑥ ⑦ 都是同一批节点（即 01 树的 ① ② ③），整段搬入、无映射；这两处的活是长干枝，并对 ⑤ ⑥ ⑦ 逐

级判定其落位，见 §2.2。

- 基数：**可 N:1，不可 1:N**——多条上游叶节点可汇入同一个下游节点；一条上游叶节点只能被一个下游节点承接。
- 依据：为什么取“最深可用原子”、为什么是 N:1——见 §1.2 依据三。

「承接」一词在本链上指两件事，勿混：本节与 §1.2 依据三说的承接，指**承接规则**——定义域只取上游叶、基数可 N:1 不可 1:N，照此 01 → 02 不是承接。另有一件叫**承接记录**的，是某张业务文档由哪张业务文档收下的逐对对应记录，由各任务写在需求侧产品数据（01）上——它是一份备档，不含映射语义，不受本规则约束（例见 biz01abr 的承接记录）。

- 已定的承接对应：

承接	上游叶	下游落点
04 → 05	Rn 树的叶（Pn 树带同名视图；P0 同此理）	系统功能需求树的 枝梢 （⑤ 用例场景功能）
06 → 07	系统功能需求树的叶（⑥ IPO功能）	【待裁】 ——平台构成树的哪一级

“叶 → 枝梢”只是其中一例的显影：在“上游已到定义、下游只到框架”的步上（04→05），落点恰是枝梢、于是错开一级。这只是两侧“各自最深可用原子”对齐的一般结果在特定完成度差下的样子——**不推出**“上游第 N 级对应下游第 N-1 / N-2 级”这类数量关系，也不表示落点必在枝梢。

其余已定的树间对应（非承接——同一对象的两面或两态，不涉及视角转换）：

对应双方	关系
02 职责视角业务流程 ↔ 03 产品开发视角业务流程	同一批业务文档的两面 ——同一对象在 R 树里是“该角色按什么序处理它”，在 P 树里是“它属哪个构件、怎么生成”；两树经 认亲 互指，末三级同名同批——实体由 02 持有、03 带同名视图
系统功能需求树的 ① 信息系统 ↔ 平台构成树的 ① 信息系统	同一系统的两面 ：需求侧功能分解宿主 = 软件构成侧（模型/前后端组件）宿主；本设计实例 = EOS 平台（企业运营体系的信息化形态）
平台构成树的根 企业信息系统	系统功能需求树的 ① 之上的 语境容器 ——企业整体信息系统，EOS 为其一
系统功能需求树 ↔ 系统非功能需求树	同一承接系统的两面 ——功能分解与非功能约束分树组织，两棵共用 ① 信息系统 根
相关方非功能需求树 ↔ 系统非功能需求树	同一 NFR 分类体系的两态：相关方侧 NFR（定性/待澄清目标值）→ 系统侧 SysReq-NFR（量化+约束）

二、各产品数据的分解结构

2.0 原始需求（入口加工链）

原始需求是设计线的**入口产品数据**——外部材料在此被加工成相关方需求基线；本链有五个加工形态，链上长出结构。

2.0.1 五个加工形态

加工形态	载体（EOS 实例）	产出工序	结构上做了什么
原始需求	00-origin-requirement-materials/10-raw-files/（PDF/图片/邮件/纪要/已有 .md）	外部输入	—（尚在 EOS 结构之外）
文本化的原始需求	*.textualized.md	eos-ort00-textualize	只换载体，结构照搬原文
切分后的原始需求	*.chunk-{seq}.md（status=raw）	eos-ort01-chunk	长出业务对象闭环 ——按对象切分，在 01 树里落为 ① 的场景坐标
澄清后的原始需求	同上文件（status=clarified）	eos-ort02-clarify	树形不动，只消歧
规范化的相关方需求	01-eos-specified-requirements.md	eos-ort03-norm	长满 01 树 （① 业务文档 → ② 任务节点 → ③ 条目）+ 条目落位

这条链不是形态轴。 框架 / 概要 / 定义说的是**同一棵树长到多深**；本链说的是**同一份材料加工到多细**。两轴的落点也不同：切分与澄清两态树形相同（澄清只清语义），原始与文本化两态**不产树**（一个尚未进 EOS 结构、一个只换载体）——五个加工形态里只有两处动结构：切分（长出业务对象闭环）与规范化（长满 01 树）。

2.0.2 链上长出的结构

文本化原始需求 —— 一份材料 = 一份文本化文档（载体形态，结构照搬原文）
└ 业务对象闭环 —— 由切分产出（属性：所属业务域、相关方角色 = 主责角色）

切分产出的「业务对象闭环」在相关方功能需求树里是 ① 业务文档的「所属 E2E 职责」属性——切分定下这条坐标，规范化再把 ① 业务文档 → ② 任务节点 → ③ 条目补满。

「业务对象闭环」= 入口加工链的切分单位 = 职责树的 E2E 职责 级，也 = 相关方功能需求树 ① 上的属性值——同一个对象，链上称闭环、R 树里占级、01 树里作属性。

2.1 相关方需求

2.1.1 相关方功能需求

相关方功能需求

- └ 业务文档 ─── ① 根 所属业务域 × 所属主责角色 × 所属E2E职责 × 所属产品类型 × 所属产品构件（五个坐标作为本级的属性）
 - └ 任务节点 ─── ② 该业务文档开发所对应任务之步骤
 - └ 用例场景 ─ ③ 任务节点中的一个用例场景（需求条目 spr-XXX）

一级节点即根：业务文档，其**所属业务域 / 所属主责角色 / 所属E2E职责 / 所属产品类型 / 所属产品构件**以**属性**随节点携带、不各占一级——这五个坐标是本份数据的**索引 / 分片轴**，不是归集级。它到方案侧对应 Rn / Pn 树 ⑤ 的**文档开发任务**，见 §2.2。

五个坐标分属两棵树：所属业务域为 R / P 两树共用，落两树的 ①；所属主责角色与所属E2E职责是 R 树的坐标，落 Rn 树 ② ④；所属产品类型与所属产品构件是 P 树的坐标，落 Pn 树 ② ③，**所属产品构件是其所属产品类型的子节点**——两级都要给，且必须挂在同一个类型之下。

本份数据的 ① ② ③ 与 Rn / Pn 树的 ⑤ ⑥ ⑦ 是同一批节点——本份产品数据是按文档平铺的一棵，R / P 树是按角色 / 场景 / 产品归拢的一棵，见 §2.2。末三级的实体由 02 持有，03 带同名视图。

条目是叶：叶 = ③ 一个用例场景（spr-XXX）；② 任务节点与 ① 业务文档是条目的**归集级**。

2.1.2 相关方非功能需求

相关方非功能需求

- └ 一级类目 ─── ① 质量特性 / 环境适应性 / 可实现性（面向哪类非功能关注）
 - └ 分类维度 ─── ② 该级类目下的分类维度（示例：系统性能、信息安全性、系统可靠性）
 - └ 具体需求 ─ ③ 某分类维度下的具体非功能需求（需求描述 + 指标口径 / 目标值）

2.2 业务流程

业务流程可以有 R、P 两个视角树（平台自身为 R0 / P0，各业务为 Rn / Pn），分装两份文档：02 职责视角业务流程概要装 R 树、03 产品开发视角业务流程概要装 P 树。定义（04）只做职责视角。

适用范围：R / P 这一对视角是「把业务信息化」这类产品特有的——业务与产品同源（产品即该业务的信息化形态），才有「从职责视角转到产品开发视角」这一步。实物产品的业务与产品不同源，不设这一对视角。

详细定义的落点是界面：菜单、窗口标签页、表单标签页、功能操作项四级都挂在 R 树上。故定义（04）只做职责视角；产品开发视角止于概要（03），其详细定义不在本树的定义里。

⑤ ⑥ ⑦ 由相关方需求直接给出：这三级的节点就是 01 树的 ① ② ③——⑤ 业务文档、⑥ 任务节点、⑦ 用例场景。从 01 到 02、从 02 到 03，这两步都不做映射、不做分配，三级的节点整段搬入；两步的活是长干枝，并对 ⑤ ⑥ ⑦ 逐级判定其落位。末三级的实体由 02 持有，03 带同名视图。

干枝由同一批属性落地：业务文档带的五个坐标，到 R 树里落成 ① 业务域 / ② 相关方角色 / ④ E2E职责，到 P 树里落成 ① 业务域 / ② 产品类型 / ③ 产品构件类型——**属性给值，组装定节点**。R 树 ③ 角色职责不出自属性，随职责视角业务流程概要演进。

⑤ **业务文档 = 业务输出的产品数据**，对应本文档的开发任务——节点名取物件（业务文档），任务名取它要开发的那份文档；这份开发任务在树里就是 ⑥ 任务节点。

枝梢与叶：⑥ **任务节点 = 枝梢**（框架的最深一级），⑦ **用例场景 = 叶**（概要的末级）。框架到 ⑥ 为止，概要到 ⑦ 为止。

文档正文不占树位——它由任务节点承载，不另设一级。

文档序列不占树位——该 E2E职责处理这批业务文档的序随 E2E职责携带，不另立一级；同一批文档在 P 树里另按构件归组，序记在 ④ 构件开发WBS（包）。

2.2.1 业务运行职责业务流程（Rn 树）

职责业务

- └ 业务域 ———— ① 归属域
 - └ 相关方角色 ———— ② 主责角色（业务对象生命周期主推动者）
 - └ 角色职责 — ③ 角色承担的职责清单，是多个E2E职责的聚类。
 - └ E2E职责 ———— ④ 一个端到端的职责业务，分执行类（从接收需求到需求被确认得到满足）与管理类（PCA）
 - └ 业务文档 — ⑤ 业务输出的产品数据，对应本文档开发任务
 - └ 任务节点 — ⑥ 文档开发任务的多个步骤，枝梢节点
 - └ 用例场景 — ⑦ 是叶

2.2.2 产品开发业务流程（Pn 树）

Pn 树是引擎平台之外的产品——各业务域的构件产品数据沿产品开发视角（PL）的分解。

输出产品业务

- └ 业务域 ———— ① 归属域
 - └ 产品类型 ———— ② 部门级的端到端业务输出
 - └ 产品构件类型 — ③ 科室级的端到端业务输出
 - └ 构件开发WBS（包）— ④ 文档序列
 - └ 业务文档 — ⑤ 业务输出的产品数据，对应本文档开发任务
 - └ 任务节点 — ⑥ 文档开发任务的多个步骤，枝梢节点
 - └ 用例场景 — ⑦ 是叶

2.2.3 业务配置职责业务流程（R0 树）

职责业务

- └ 业务域 ———— ① 流程与IT（平台自身域——单域，不分业务域）

- └ 相关方角色 —— ② 业务配置角色：业务配置人员 / 平台治理角色（业务架构管理员、数据治理员等）
 - └ E2E职责 - ③ 一个端到端的职责业务（bpl-eng 的能力场景 / 平台资产治理业务）
 - └ 业务文档 - ④ 业务输出的产品数据，对应本文档开发任务
 - └ 任务节点 - ⑤ 文档开发任务的多个步骤，枝梢节点
 - └ 用例场景 - ⑥ 是叶

2.2.4 引擎平台开发业务流程（PO 树）

输出产品业务（单分支——只有配置单元）

- └ 业务域 —— ① 业务域（IT与信息化）
 - └ 产品类型 —— ② 产品类型（EOS平台）
 - └ 构件 —— ③ 构件（引擎，如 流程引擎 @engine-flow）
 - └ 配置单元 —— ④ 配置单元（CU，@xx-cu-zzz）
 - └ 配置信息组 —— ⑤ 配置信息组

2.3 系统需求

2.3.1 系统功能需求

SR-F 系统功能需求树

- └ 信息系统 —— ① 承接业务流程的信息系统——业务流程可能被不同的信息系统承接，故先按承接系统分组（可有多个）
 - └ 系统用例功能簇 - ② 对系统用例功能类的再聚类
 - └ 系统用例功能类 - ③ 按照某个维度对系统用例功能的聚类
 - └ 系统用例功能 - ④ 实现同一功能的不同实现方式
 - └ 用例场景功能 - ⑤ 将用例场景（用户视角）转换为系统视角的功能
 - └ IPO功能 - ⑥ 系统对输入的响应

注·③ 按系统职责聚类：③ 系统用例功能类按**系统职责**聚类——回答"这个系统用例功能在系统里主要负责什么、由哪类机制承接"，**不是**"它属于哪个业务域、哪个业务对象"。② 是 ③ 的再聚类（同一维度、粗一档）。① 信息系统只做"哪套系统承接业务"的分群，不替代职责聚类。

2.3.2 系统非功能需求

SR-N 系统非功能需求树

- └ 信息系统 —— ① 承接系统（非功能需求同样按承接系统挂）
 - └ 一级类目 —— ② 质量特性 / 环境适应性 / 可实现性（面向哪类非功能关注）
 - └ 分类维度 —— ③ 该级类目下的分类维度（示例：系统性能、信息安全、系统可靠性）
 - └ 具体系统非功能需求 - ④ 某分类维度下的 SysReq-NFR 末级（量化指标+统计口径+验证方法）

2.4 平台构成

企业信息系统 —— 根 企业整体的信息系统（目标语境）
└ 信息系统 —— ① 一个具体信息系统（EOS 平台 = 企业运营体系的信息化形态）
 └ 模型 —— ② 模型（目录性质：对应代码的文件夹）
 └ 前后端组件 —— ③ 枝梢节点（前端组件 / 后端组件）

2.5 配置方案

【待定义】结构树待定义——旧平台对象（功能表单 / 流程定义 ...）的产物归属在此收口。

2.6 前后端组件方案

2.6.1 前端组件方案

【待定义】结构树待定义。

2.6.2 后端组件方案

【待定义】结构树待定义。